

오목 게임 프로젝트

작성자 :
강이영, 박정환

목차

1. 개요
2. 개발 환경
3. 기능 정의서
4. 순서도
5. 주요 서비스 화면
6. 보완할 점 & 소감

서비스 과제

오목 게임 프로그램 개발

목적

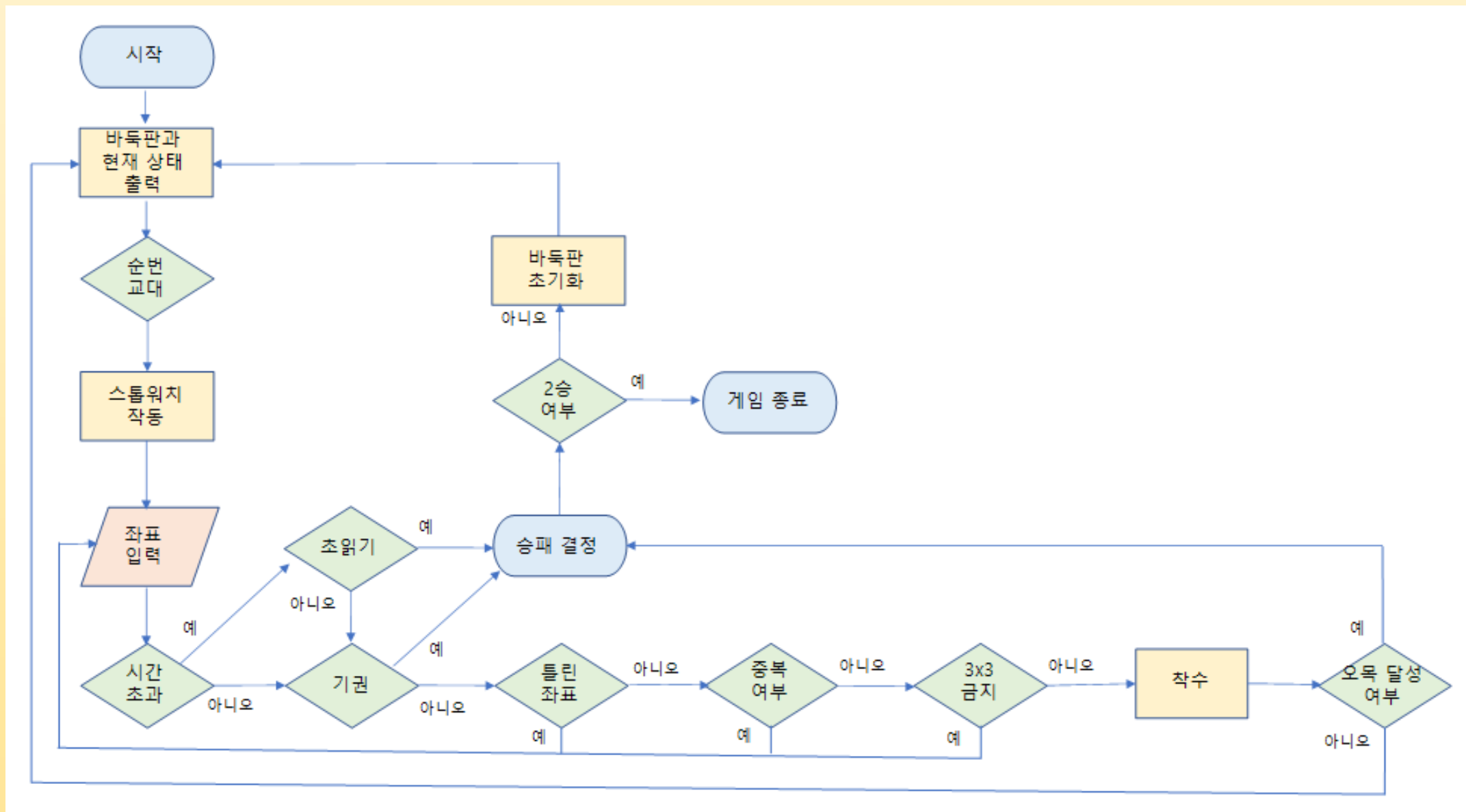
- 두 사람이 번갈아 가며 즐길 수 있는 오목 게임을 구현

주요 기능

- 순번 교대
- 제한시간, 초읽기 기능
- 3x3 금지 기능
- 3판 2승제

- 운영체제 : 윈도우 10 Pro 64비트 (10.0, 빌드 19045)
- 메모리 : 8GB
- JDK : 1.8
- IDE : eclipse 4.8.0 (확장 프로그램: ANSI Escape Codes) / IntelliJ IDEA 2024.3.2.2(Ultimate Edititon)

번호	기능	기능 설명
1	바둑판 출력	바둑판과 현재 놓인 돌의 상태를 출력한다.
2	순번 교대	흑돌과 백돌의 순서를 바꾼다.
3	스톱워치	제한 시간과 초읽기를 계산한다.
4	좌표 입력	돌을 놓을 위치를 지정한다.
5	시간 초과와 초읽기	제한 시간 5분, 초읽기 3회 30초 부여. 제한 시간 소진 후, 초읽기 횟수를 모두 사용하면 패배 처리.
6	금지 규칙	• 이미 돌을 놓은 곳에 좌표를 입력하는 경우 재입력 • 좌표를 입력하지 않거나, 범위를 벗어날 경우 재입력 • 3x3 규칙에 해당하는 곳에 좌표를 입력한 경우 재입력
7	착수	지정한 위치에 돌을 놓음.
8	승리 조건	• 오목 달성 • 상대방이 기권 • 상대방이 초읽기 횟수까지 다 소진
9	색상 표시	• 가시성을 높이기 위해 메시지에 색상 사용. (e. g. 승리 메시지: 노란색, 오류 메시지: 빨간색)



① 시작 화면

```

-----
: 오목게임 :
----- ver 1.0
흑돌 사용자 이름을 입력해주세요.
aaaa
백돌 사용자 이름을 입력해주세요.
bbbb
aaaa, bbbb님 환영합니다.
오목 게임은 3판 2선승제로 2번 먼저 이기시면 게임에서 최종 승리합니다.
시작하시려면 Enter를 눌러주세요.

```

1번째 게임을 시작합니다.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0																			
1																			
2																			
3																			
4																			
5																			
6																			
7																			
8																			
9																			
10																			
11																			
12																			
13																			
14																			
15																			
16																			
17																			
18																			

aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요. (000을 누르면 기권 패배로 처리합니다.)남은 시간 : **흑** 5분 0초 / **백** 5분 0초

9,9

② 게임 중 화면

bbbb님의 차례입니다. 좌표를 x,y 형태로 입력해주세요. (000을 누르면 기권 패배로 처리합니다.)

남은 시간 :  4분 42초 /  4분 8초

- 바둑판에 착수한 돌 표시
- 순번(흑/백) 번갈아 가며 경기
- 남은 시간 표시

③ 승패 조건

- 기권으로 인한 패배, 상대방의 승리

③ 승패 조건

18 -

aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요.(000을 누르면 기권 패배로 처리합니다.)

남은 시간 : 흑 1분 43초 / 백 4분 25초

9,9

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
0																			
1																			
2																			
3																			
4																			
5																			
6																			
7																			
8																			
9																			
10																			
11																			
12																			
13																			
14																			
15																			
16																			
17																			
18																			

오목 완성.
이번 경기 승리.

aaaa님이 이겼습니다.
-게임종료-
최종 승자는 aaaa님입니다.

2승을 먼저한 흑이
최종 승리

- 오목 완성으로 승리
- 2승을 먼저 거둔 흑이 최종 승리

5. 주요 서비스 화면

③ 승패 조건

aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요. (000을 누르면 기권 패배로 처리합니다.)

남은 시간 : ██ 0분 30초 / ██ 0분 30초

제한시간이 지났습니다. 이제부터 30초 타이머가 작동됩니다.

타이머에 3번 걸리면 자동으로 패배합니다.

남은 횟수는 1번 입니다.

0,0

30초 타이머에 걸렸습니다.

남은 횟수는 0번 입니다.

bbbb님이 이겼습니다.

현재 스코어= 0 : 1

=====

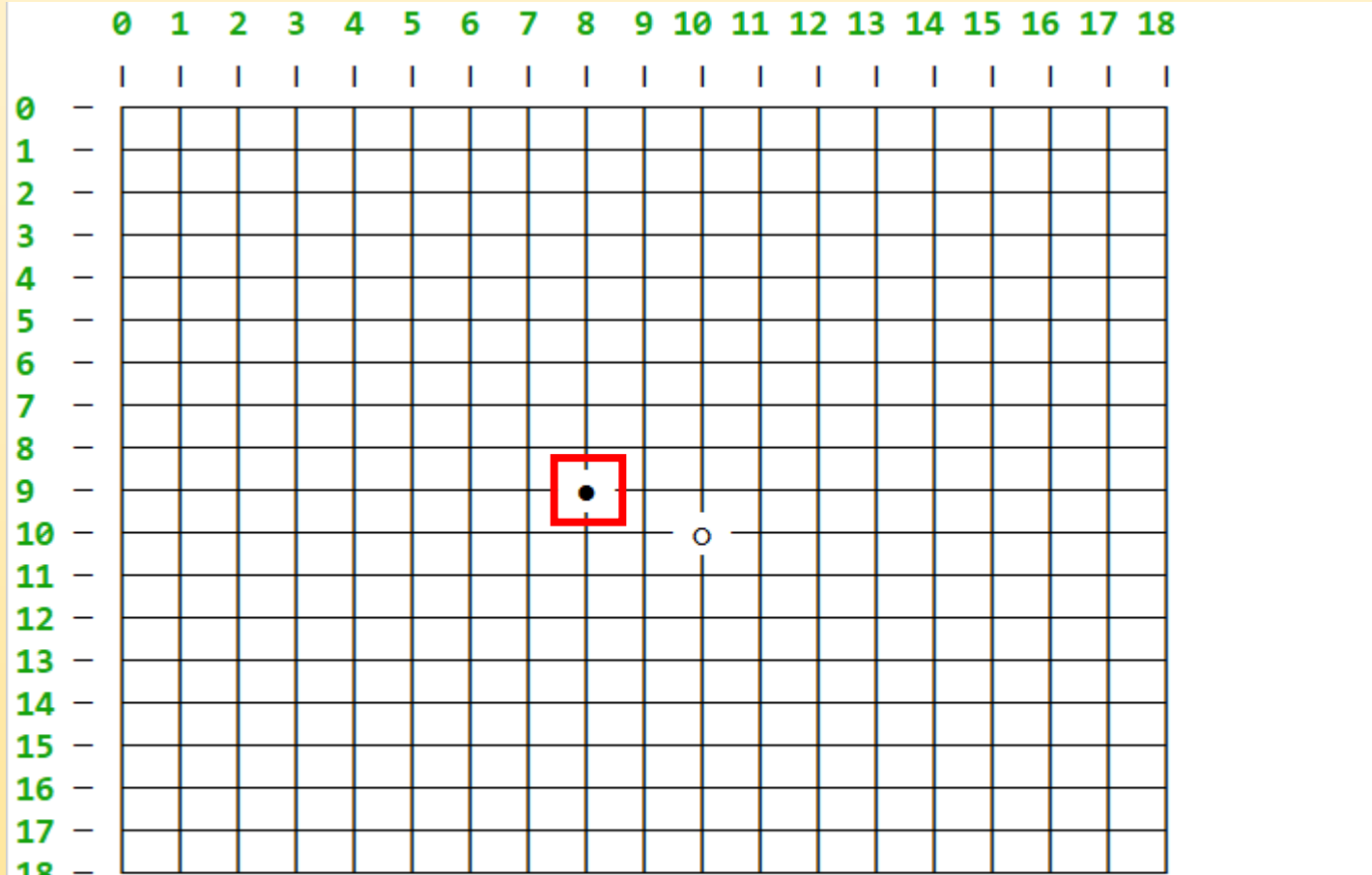
다음 게임으로 넘어갑니다.

계속하려면 엔터를 눌러주세요.

착수가 되지 않은 채
초읽기 소진으로 인한 패배

- 남은 시간 소진 후,
초읽기 기회 3번도 다 사용하여 패배.
상대방의 승리.

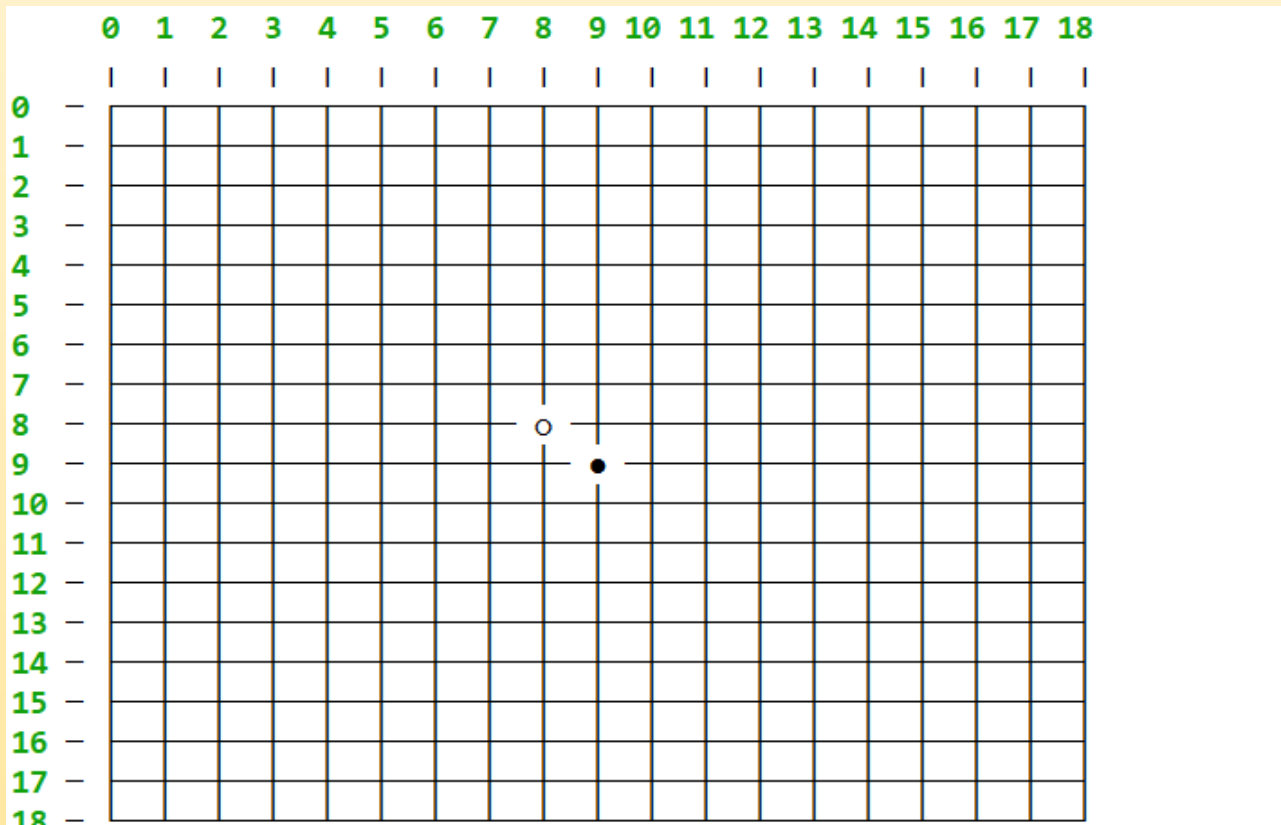
④ 금지 규칙



aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요. (000을 누르면 기권 패배로 처리합니다.)
남은 시간 : **흑** 4분 56초 / **백** 4분 57초
9,8
중복입니다. 다시 입력해주세요

- 중복 금지: 돌이 있는 곳에 착수할 수 없다.

④ 금지 규칙



aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요. (000을 누르면 기권 패배로 처리합니다.)

남은 시간 : **흑** 4분 53초 / **백** 4분 58초

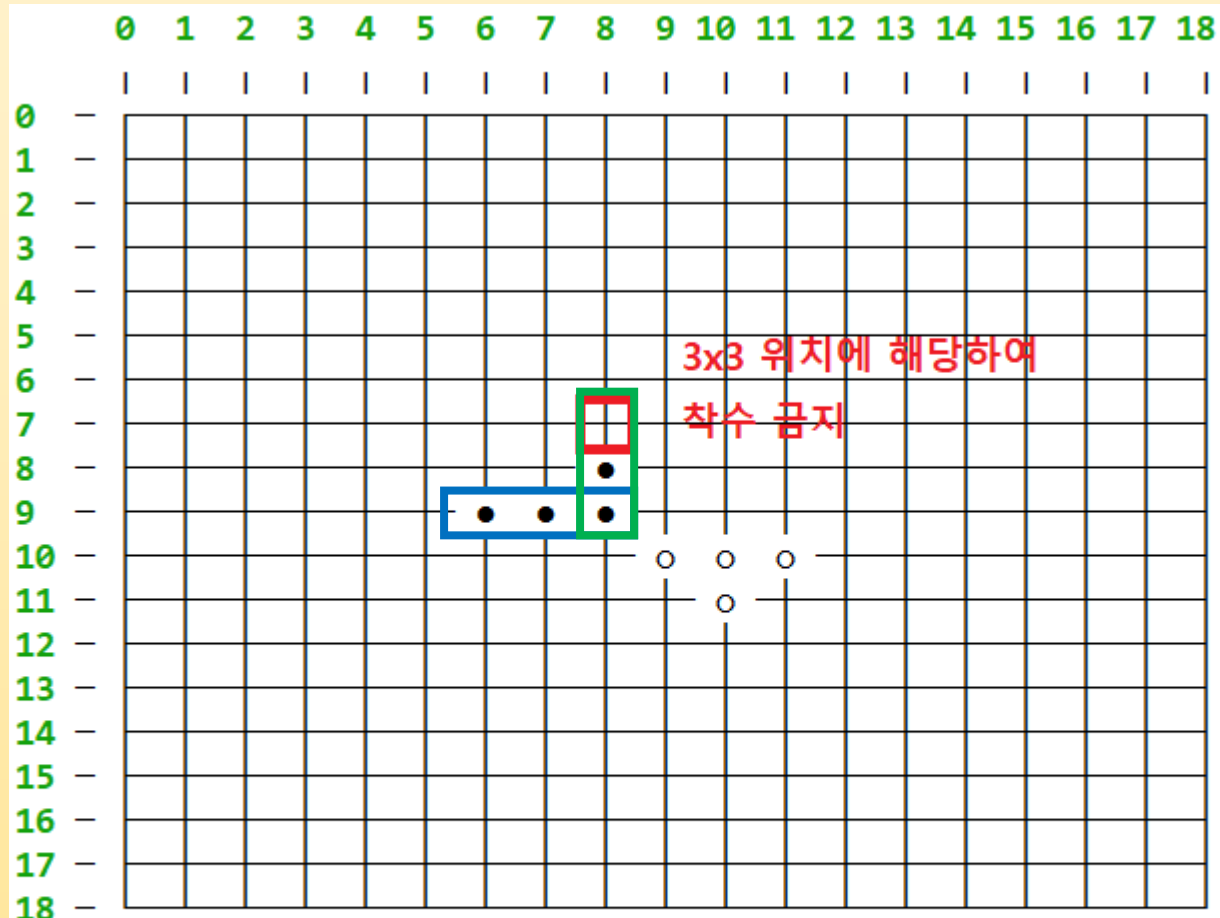
20,20

오목판 범위를 벗어납니다. 다시 입력해주세요

좌표를 입력하지 않았습니다. 다시 입력해주세요

- 좌표 미입력, 바둑판을 벗어나는 값 금지

④ 금지 규칙



- 3x3 금지: 가로, 세로, 대각선으로 연속해서 놓인 돌이 3개인 선이 2개여서는 안된다.

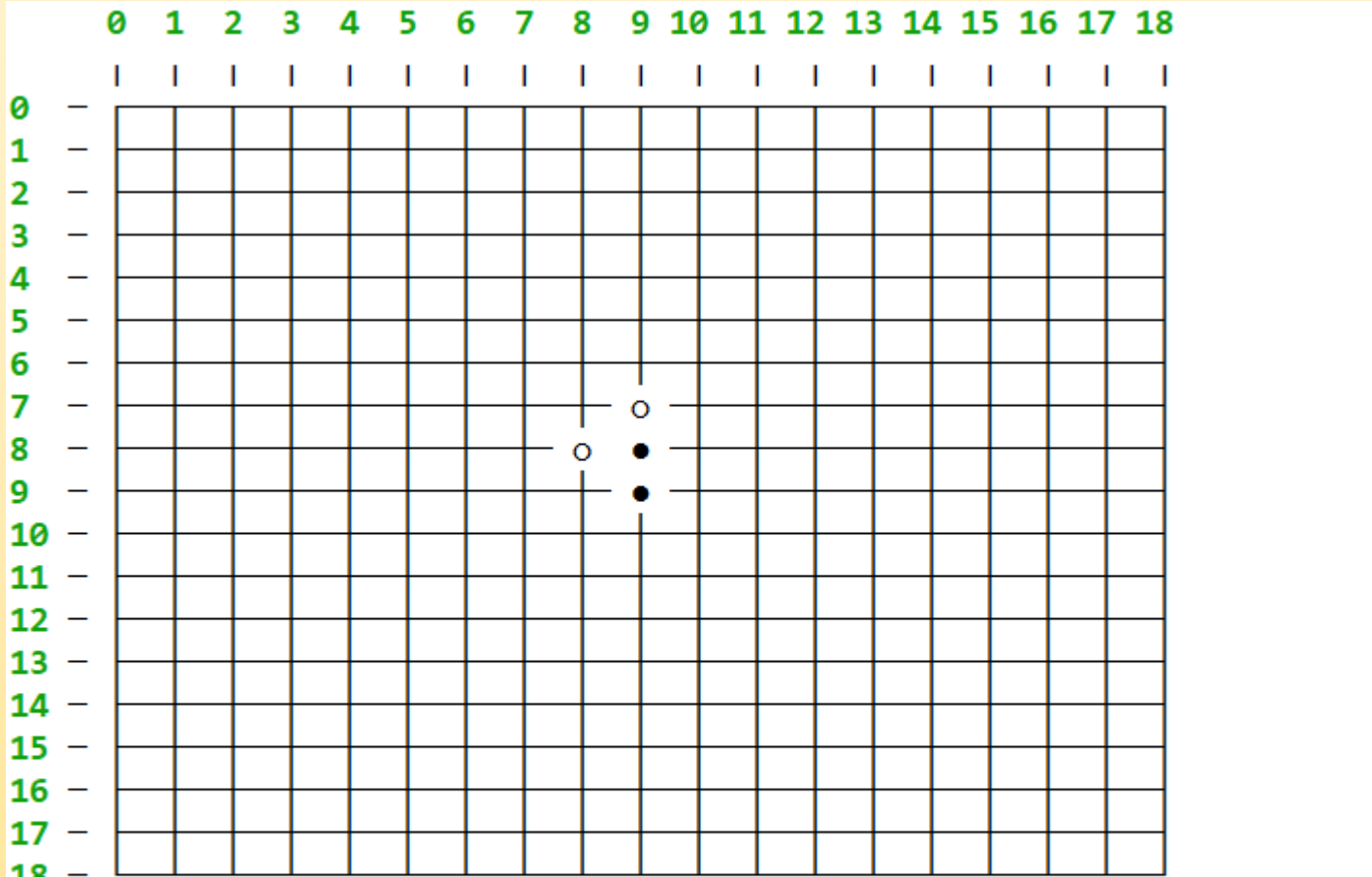
aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요.(000을 누르면 기권 패배로 처리합니다.)

남은 시간 : 4분 2초 / 4분 37초

7,8

삼삼규칙에 어긋납니다. 다시 입력해주세요

⑤ 제한시간과 초읽기



aaaa님의 차례입니다. 좌표를 x,y 형태로 입력해주세요.(000을 누르면 기권 패배로 처리합니다.)

남은 시간 : 흑 0분 30초 / 백 0분 9초

제한시간이 지났습니다. 이제부터 30초 타이머가 작동됩니다.

타이머에 3번 걸리면 자동으로 패배합니다.

남은 횟수는 3번 입니다.

- 초읽기 기능: 제한시간을 다 쓰면, 30초의 초읽기를 3번 쓸 수 있다.
- 30초 이내에 돌을 놓으면, 해당 횟수의 초읽기는 유지. 다음 차례 때 다시 30초부터 시작한다.
- 30초가 지나면, 초읽기 횟수 차감.
- 초읽기 횟수 3회를 모두 소진하면 패배. (승패 조건의 마지막 조건 참조)
- 게임 내에서는 "타이머"로 표기.

박정환

- 시간 작동과 좌표 입력을 동시에 진행하지 못하는 점을 보완할 필요가 있음.
이는 두 개 이상의 클래스(class)를 붙이거나, 다른 멀티 스레드를 작동시키는 기능이 더 필요.
 - 삼삼규칙의 경우 상대방의 돌이 옆에 있다거나 가장자리에 위치함으로써 오목 달성이 어려운 경우까지 처리할 수 없었음.
 - GUI에 대한 지식이 없는 상태에서 바둑판과 바둑알을 놓는 구상을 실현한 것은 만족.
-

강이영

- 처음에 메인 메서드 부분을 생각나는 대로 작성함.
구조가 명확하게 보이지 않았고, 그에 따라 새로운 메서드를 작성할 때마다 많이 헤맸으며, 시간 역시 많이 소요됨.
코드 간략화를 하면서 코드 구조를 파악하기 쉽게 변경하긴 했지만 오류를 많이 고쳐야 했음.
비효율적인 코드 작성 절차를 고쳐야겠다고 생각함.
- 처음에 오목의 규칙과 개념을 오해한 상태에서 작성한 코드가 오목 규칙을 제대로 반영하지 못해, 코드 작성에 더 많은 시간이 소요됨.
이후 이 점을 보완하여 규칙을 반영한 것은 협업의 성과.

감사합니다

<https://github.com/kangeyoung/Omok/tree/jpminlak>

부록

5. 주요 서비스 화면

⑥ 주요 소스 코드

```
static Scanner s = new Scanner(System.in); 5 usages
public static final String ANSI_RESET = "\033[0m"; // Text Reset 22 usages
public static final String ANSI_GOLD = "\033[0;33m"; // GOLD 3 usages
public static final String ANSI_BLACK_BOLD = "\033[1;30m"; // BLACK 2 usages
public static final String ANSI_CYAN_BOLD = "\033[1;36m"; // CYAN 1 usage
public static final String ANSI_WHITE_BOLD = "\033[1;37m"; // WHITE 2 usages
public static final String ANSI_BLACK_BACKGROUND = "\u001B[40m"; // BLACK 2 usages
public static final String ANSI_RED_BOLD_BRIGHT = "\033[1;91m"; // RED 6 usages
public static final String ANSI_WHITE_BACKGROUND_BRIGHT = "\033[0;107m"; // WHITE 2 usages
public static final String ANSI_GREEN_BOLD = "\033[1;32m"; // GREEN 9 usages
public static final String ANSI_GOLD_BOLD = "\033[1;33m"; // GOLD 5 usages
// 프로젝트 > Properties > Resource > Text file encoding- other: UTF-8 변경 필요
static int[][] goBoard = new int[19][19]; // 흑: 1, 백: 2 12 usages
static int startTime; // 시작 시간 2 usages
static int elapsedTime; // 경과 시간 3 usages
// 배열의 인덱스 0 = 흑, 1 = 백
static String[] name = new String[2]; 4 usages
static int[] Timeout = {3, 3}, 5 usages
    Time = {300, 300}, // 제한시간 5분 11 usages
    win = {0, 0}; 8 usages
static boolean[] Timer = {false, false}; // 5분 지나면 30초 타이머 on(=true) 5 usages
```

- 메인 메서드

⑥ 주요 소스 코드

```

28 // 게임시작
29 public static void main(String[] args) {
30     // TODO Auto-generated method stub
31     System.out.println("-----");
32     System.out.println(": " + ANSI_GOLD_BOLD + "오목게임 " + ANSI_RESET + " :");
33     System.out.println("----- ver 1.0");
34     System.out.println("흑돌 사용자 명을 입력해주세요.");
35     name[0] = s.nextLine();
36     System.out.println("백돌 사용자 명을 입력해주세요.");
37     name[1] = s.nextLine();
38     System.out.println(nameColor(0) + ", " + nameColor(1) + "님 환영합니다.");
39     System.out.println("오목 게임은 3판 2선승제로 2번 먼저 이기시면 게임에서 최종 승리합니다.");
40     System.out.println("시작하시려면 " + ANSI_CYAN_BOLD + "Enter" + ANSI_RESET + "를 눌러주서");
41     s.nextLine();
42     for (int i = 1; win[0] < 2 && win[1] < 2; i++) {
43         if (i > 1) {
44             resetSetting();
45         }
46         System.out.println(i + "번째 게임을 시작합니다.");
47         printBoard();
48         while (true) {
49             if (gameStart(0)) {
50                 break;
51             }
52             if (gameStart(1)) {
53                 break;
54             }
55         }
56     }
57     if (win[0] == 2) {
58         System.out.println(ANSI_GOLD_BOLD + "-게임종료- \n최종 승자는 " + nameColor(0) + AN.
59     } else {
60         System.out.println(ANSI_GOLD_BOLD + "-게임종료- \n최종 승자는 " + nameColor(1) + AN.
61     }
62 }

```

• 메인 메서드

⑥ 주요 소스 코드

```

64 // 오목판 출력
65 public static void printBoard() {
66     System.out.println(
67         ANSI_GREEN_BOLD + "      0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18" + ANSI_RESET);
68     System.out.println("      |  |  |  |  |  |  |  |  |  |  |  |  |  |  |  |");
69     for (int i = 0; i < 19; i++) {
70         for (int j = 0; j < 19; j++) {
71             if (goBoard[i][j] == 0) {
72                 if (i == 0) {
73                     if (j == 0) {
74                         System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + "  -  |"); // -, |, |, |, |, |, |, |
75                         // 는 특수기호
76                     } else if (j == 18) {
77                         System.out.println("-");
78                     } else {
79                         System.out.print("-");
80                     }
81                 } else if (i == 18) {
82                     if (j == 0) {
83                         System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + "  -  |");
84                     } else if (j == 18) {
85                         System.out.println("-");
86                     } else {
87                         System.out.print("-"); //  0  1  2  3  4  5  6  7  8  9  10  11  12  13  14  15  16  17  18
88                     }
89                 } else {
90                     if (j == 0) {
91                         if (i <= 9) {
92                             System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + "  -  |");
93                         } else {
94                             System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + "  -  |");
95                         }
96                     } else if (j == 18) {
97                         System.out.println("-");
98                     } else {
99                         System.out.print("-");
100                    }
                }
            }
        }
    }
}

```

• 바둑판 구현

매개변수: 없음
리턴값: 없음

⑥ 주요 소스 코드

```
102         } else if (goBoard[i][j] == 1) {
103             if (j == 0) {
104                 if (i <= 9) {
105                     System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + " - ● ");
106                 } else {
107                     System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + " - ● ");
108                 }
109             } else if (j == 18) {
110                 System.out.println(" ●");
111             } else {
112                 System.out.print(" ● ");
113             }
114         } else if (goBoard[i][j] == 2) {
115             if (j == 0) {
116                 if (i <= 9) {
117                     System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + " - ○ ");
118                 } else {
119                     System.out.print(ANSI_GREEN_BOLD + i + ANSI_RESET + " - ○ ");
120                 }
121             }
122             } else if (j == 18) {
123                 System.out.println(" ○");
124             } else {
125                 System.out.print(" ○ ");
126             }
127         }
128     }
129 }
130 }
```

- 바둑판 구현에 놓은 돌 표시: 입력받은 배열값에 따라 바둑판 출력

⑥ 주요 소스 코드

```

150 // 착수
151 public static boolean gameStart(int num) {
152     int num2;
153     if (num == 0) {
154         num2 = 1;
155     } else {
156         num2 = 0;
157     }
158     System.out.println(nameColor(num) + "님의 차례입니다. 좌표를 x,y 형태로 입력해주세요."
159         + "(000을 누르면 기권 패배로 처리합니다.)"); // 기권 번호 안내
160     System.out.println("남은 시간 : " + ANSI_BLACK_BACKGROUND + ANSI_WHITE_BOLD + "흑" +
161         + Time[0] / 60 + "분 " + Time[0] % 60 + "초 / "
162         + ANSI_WHITE_BACKGROUND_BRIGHT + ANSI_BLACK_BOLD + "백" + ANSI_RESET + "
163         + Time[1] / 60 + "분 " + Time[1] % 60 + "초"); // 남은 시간
164     int result;
165     if (Timer[num]) { // 5분 제한시간 끝났을 때
166         System.out.println(ANSI_RED_BOLD_BRIGHT + "제한시간이 지났습니다. 이제부터 30초 타이머가 작동
167             "\n남은 횟수는 " + Timeout[num] + "번 입니다." + ANSI_RESET);
168         stopwatch(0);
169         result = coordinateIn(num + 1);
170         if (stopwatch(num + 1)) {
171             Timeout[num]--;
172             System.out.println(ANSI_RED_BOLD_BRIGHT + "30초 타이머에 걸렸습니다." +
173                 "\n남은 횟수는 " + Timeout[num] + "번 입니다.\n 2초 후에 다음 플레이어의 턴으로 넘어갑니다.");
174             try {
175                 Thread.sleep(2000);
176             } catch (InterruptedException e) {
177                 e.printStackTrace();
178             }
179         }
180         Time[num] = 30; // 30초 타이머 초기화
181     } else {
182         stopwatch(0);
183         result = coordinateIn(num + 1);
184         Timer[num] = stopwatch(num + 1);
185         if (Timer[num]) { // 5분 제한시간 끝났을 때
186             Time[num] = 30; // 타이머 30초로 설정

```

- 승패조건, 제한시간, 초읽기(타이머) 등

- 착수 및 승리조건에 따라 리턴값 리턴

매개변수: int num(흑 0 과 백 1 구분용)

리턴값: boolean true(세트 종료) false

⑥ 주요 소스 코드

```
187         }
188     }
189     if (Timeout[num] == 0) { // 타임아웃
190         win[num2]++;
191         System.out.println(nameColor(num2) + ANSI_GOLD + "님이 이겼습니다.");
192         return true;
193     }
194     printBoard();
195     if (result == -1) {
196         win[num]++;
197         System.out.println(nameColor(num) + ANSI_GOLD + "님이 이겼습니다.");
198         return true;
199     } else if (result == -2) {
200         win[num2]++;
201         System.out.println(nameColor(num2) + ANSI_GOLD + "님이 이겼습니다.");
202         return true;
203     }
204     return false;
205 }
```

- 승패조건, 제한시간, 초읽기(타이머) 등

⑥ 주요 소스 코드

```

208 // 좌표입력
209 public static int coordinateIn(int bw) {
210     while (true) {
211         int[] coor = new int[2];
212         String in = s.nextLine();
213         if (in.isEmpty()) { // 좌표 입력 안 했을때
214             System.out.println(ANSI_RED_BOLD_BRIGHT + "좌표를 입력하지 않았습니다. 다시 입력해주세요");
215             continue;
216         } else if (in.equals("000")) { // 기권
217             return -2;
218         } else {
219             int k = 0;
220             for (int i = 0; i < in.length(); i++) { // 사용자에게 받은 문자열을 좌표로 변환
221                 if (in.charAt(i) >= 48 && in.charAt(i) <= 57) {
222                     coor[k] *= 10;
223                     coor[k] += in.charAt(i) - 48;
224                 } else {
225                     k++;
226                 }
227             }
228             if (coor[0] > 18 || coor[1] > 18) { // 오목판을 벗어나는 숫자 금지
229                 System.out.println(ANSI_RED_BOLD_BRIGHT + "오목판 범위를 벗어납니다. 다시 입력해주세요");
230                 continue;
231             }
232             if (goBoard[coor[0]][coor[1]] == 1 || goBoard[coor[0]][coor[1]] == 2) {
233                 System.out.println(ANSI_RED_BOLD_BRIGHT + "중복입니다. 다시 입력해주세요" + ANSI_RED_BOLD_BRIGHT);
234                 continue;
235             } else {
236                 goBoard[coor[0]][coor[1]] = bw;
237                 if (prohibitDoubleThree(coor[0], coor[1], bw)) { // 삼삼 금지
238                     goBoard[coor[0]][coor[1]] = 0;
239                     System.out.println(ANSI_RED_BOLD_BRIGHT + "삼삼규칙에 어긋납니다. 다시 입력해주세요");
240                     continue;
241                 }
242                 if (victoryFive(coor[0], coor[1], bw)) {
243                     return -1; // 오목 달성
244                 }
245             }

```

```

246         }
247         break;
248     }
249     return 0;
250 }

```

- 입력 받은 좌표를 저장
- 승리 조건에 따라 리턴값 리턴

매개변수: int bw(흑 1과 백 2 구분용)

리턴값: int -1(본인 승리), -2 (상대편 승리), 0(턴 종료)

⑥ 주요 소스 코드

```
431 // 5분 스톱워치
432 public static boolean stopwatch(int onOff) {
433     if (onOff == 0) { // 타이머 켜기
434         startTime = (int) System.currentTimeMillis() / 1000;
435     }
436     if (onOff == 1 || onOff == 2) { // 타이머 끄기
437         int endTime = (int) System.currentTimeMillis() / 1000;
438         elapsedTime = endTime - startTime;
439         int num = onOff - 1;
440         if (Timer[num]) {
441             Time[num] = 30 - elapsedTime;
442         } else {
443             Time[num] -= elapsedTime;
444         }
445         if (Time[num] <= 0) {
446             Time[num] = 0;
447             return true;
448         }
449     }
450     return false;
451 }
```

스톱워치

- 입력하기 전 시간을 기록하고, 입력한 후의 시간을 다시 기록하여 앞의 시간과 비교.
- 경과 시간을 남은 제한시간이나 초읽기 시간에서 차감.
- 제한시간 5분과 그 이후 30초 타이머 구현

매개변수: int onOff(타이머시작 0 혹은 1 백 2)
리턴값: boolean true(제한시간 지남), false

⑥ 주요 소스 코드

```

271 // 연속하는 돌의 개수
272 public static int[][] stoneConsecutive(int i, int j, int num, int bw) {
273     int stoneHorizontal = 0; // 가로로 연속으로 같은 색의 돌일 경우 집계하는 변수
274     int stoneVertical = 0; // 세로로 연속으로 같은 색의 돌일 경우 집계하는 변수
275     int stoneDiagonalRight = 0; // 오른쪽 대각선(/)으로 연속으로 같은 색의 돌일 경우 집계하는 변수
276     int stoneDiagonalLeft = 0; // 왼쪽 대각선(\)으로 연속으로 같은 색의 돌일 경우 집계하는 변수
277     int[][] conse = new int[4][3];
278     for (int h = j - num + 1; h < j + num; h++) {
279         if (h < 0 || h > 18) {
280             continue;
281         }
282         if (goBoard[i][h] == bw) {
283             stoneHorizontal++;
284         } else {
285             stoneHorizontal = 0;
286         }
287         if (stoneHorizontal == num) {
288             conse[0][0] = stoneHorizontal;
289             conse[0][1] = i; // 시작점
290             conse[0][2] = h - num + 1;
291             break;
292         }
293     }
294     for (int h = i - num + 1; h < i + num; h++) {
295         if (h < 0 || h > 18) {
296             continue;
297         }
298         if (goBoard[h][j] == bw) {
299             stoneVertical++;
300         } else {
301             stoneVertical = 0;
302         }
303         if (stoneVertical == num) {
304             conse[1][0] = stoneVertical;
305             conse[1][1] = h - num + 1; // 시작점
306             conse[1][2] = j;
307             break;
308         }

```

연속하는 돌의 개수 세기

- 입력받은 좌표에서 몇 개의 돌이 연속해서 배치되어있는 지 계산
- 가로, 세로, 오른쪽 대각선(/), 왼쪽 대각선(\) 으로 집계
- 시작점 정보 배열 리턴

매개변수: int i(goBoard 배열 row index),
 int j(goBoard 배열 col index),
 int num(연속하는 돌의 개수), int bw(흑과 백 구분)

리턴값: int[][] row: 가로, 세로, 오른쪽 대각선, 왼쪽 대각선
 col: 집계값, 시작점(goBoard 배열 row index, goBoard 배열 col index)

⑥ 주요 소스 코드

```

310 int s = i - num + 1; // i좌표 변환
311 for (int h = j - num + 1; h < j + num; h++) {
312     if (h < 0 || h > 18 || s < 0 || s > 18) {
313         s++;
314         continue;
315     }
316     if (goBoard[s][h] == bw) {
317         s++;
318         stoneDiagonalRight++;
319     } else {
320         s++;
321         stoneDiagonalRight = 0;
322     }
323     if (stoneDiagonalRight == num) {
324         conse[2][0] = stoneDiagonalRight;
325         conse[2][1] = s - num; // 시작점
326         conse[2][2] = h - num + 1;
327         break;
328     }
329 }
330 s = i + num - 1;
331 for (int h = j - num + 1; h < j + num; h++) {
332     if (h < 0 || h > 18 || s < 0 || s > 18) {
333         s--;
334         continue;
335     }
336     if (goBoard[s][h] == bw) {
337         s--;
338         stoneDiagonalLeft++;
339     } else {
340         s--;
341         stoneDiagonalLeft = 0;
342     }
343     if (stoneDiagonalLeft == num) {
344         conse[3][0] = stoneDiagonalLeft;
345         conse[3][1] = s + num; // 시작점
346         conse[3][2] = h - num + 1;
347         break;

```

```

348     }
349 }
350 return conse;
351 }

```

연속하는 돌의 개수 세기

- 리턴되는 배열값 예시

conse(예시)	집계값	시작점 row	시작점 col
stoneHorizontal	5	7	9
stoneVertical	0	0	0
stoneDiagonalRight	0	0	0
stoneDiagonalLeft	0	0	0

⑥ 주요 소스 코드

```

353 // 3x3 금지 기능
354 public static boolean prohibitDoubleThree(int i, int j, int bw) { // call 받는 변수 필요
355     int[][] DoubleThree = stoneConsecutive(i, j, 3, bw); // 가로, 세로, /, \ 로 연속으로 놓은 돌의 개수
356     int[][] conseAgain = new int[4][3]; //stoneConsecutive 다시 돌리고 저장할 배열
357     if (DoubleThree[0][0] == 3) { // 기준점 가로
358         int y = DoubleThree[0][1];
359         int x = DoubleThree[0][2];
360         for (int s = 1; s < 4; s++) {
361             conseAgain = stoneConsecutive(y, x, 3, bw);
362             for (int h = 0; h < 4; h++) {
363                 if (conseAgain[h][0] == 3 && h != 0) {
364                     return true;
365                 }
366             }
367             x++;
368         }
369     }
370     if (DoubleThree[1][0] == 3) { // 기준점 세로
371         int y = DoubleThree[1][1];
372         int x = DoubleThree[1][2];
373         for (int s = 1; s < 4; s++) {
374             conseAgain = stoneConsecutive(y, x, 3, bw);
375             for (int h = 0; h < 4; h++) {
376                 if (conseAgain[h][0] == 3 && h != 1) {
377                     return true;
378                 }
379             }
380             y++;
381         }
382     }

```

• 3x3 금지: 입력값을 토대로 3x3규칙 위반 여부 판별

매개변수: int i(goBoard 배열 row index),
int j(goBoard 배열 col index),
int bw(흑과 백 구분)

리턴값: boolean true(삼삼규칙 위반) false

⑥ 주요 소스 코드

```
383     if (DoubleThree[2][0] == 3) { // 기준점 오른쪽 대각선
384         int y = DoubleThree[2][1];
385         int x = DoubleThree[2][2];
386         for (int s = 1; s < 4; s++) {
387             conseAgain = stoneConsecutive(y, x, 3, bw);
388             for (int h = 0; h < 4; h++) {
389                 if (conseAgain[h][0] == 3 && h != 2) {
390                     return true;
391                 }
392             }
393             y++;
394             x++;
395         }
396     }
397     if (DoubleThree[3][0] == 3) { // 기준점 왼쪽 대각선
398         int y = DoubleThree[3][1];
399         int x = DoubleThree[3][2];
400         for (int s = 1; s < 4; s++) {
401             conseAgain = stoneConsecutive(y, x, 3, bw);
402             for (int h = 0; h < 4; h++) {
403                 if (conseAgain[h][0] == 3 && h != 3) {
404                     return true;
405                 }
406             }
407             y--;
408             x++;
409         }
410     }
411     return false;
412 }
```

- 3x3 금지

⑥ 주요 소스 코드

```
274 // 오목달성
275 public static boolean victoryFive(int i, int j, int bw) {
276     int[][] vic = stoneConsecutive(i, j, 5, bw);
277     for (int h = 0; h < 4; h++) {
278         if (vic[h][0] == 5) {
279             return true;
280         }
281     }
282     return false;
283 }
```

- 입력값을 토대로 오목 달성 여부 판별

매개변수: int i(goBoard 배열 row index),
int j(goBoard 배열 col index),
int bw(흑과 백 구분)

리턴값: boolean true(오목 달성), false

⑥ 주요 소스 코드

```
264 // 오목판 초기화
265 public static void resetBoard(int[][] a) {
266     for (int i = 0; i < a.length; i++) {
267         for (int j = 0; j < a.length; j++) {
268             a[i][j] = 0;
269         }
270     }
271 }
```

- 승패 결정 이후 바둑판 초기화, 세팅 초기화 및 세트 스코어 출력

```
135 // 세팅 초기화 및 세트 스코어 출력
136 public static void resetSetting() {
137     System.out.println("현재 스코어= " + win[0] + " : " + win[1] + "\n=====");
138     System.out.println("다음 게임으로 넘어갑니다.");
139     System.out.println("계속하려면 엔터를 눌러주세요."); // 다음 게임 진행 전 잠시 대기
140     s.nextLine();
141     for (int i = 0; i < 2; i++) {
142         Timeout[i] = 3;
143         Timer[i] = false;
144         Time[i] = 300;
145     }
146     resetBoard(goBoard);
147 }
```

매개변수:

resetBoard - 배열 (초기값으로 되돌릴 배열)

resetSetting - 없음

리턴값: 없음